جلسه‌ی همراه با هوش مصنوعی: MiA Cold Start

این جلسه چه چیزی را می‌سنجد

یک جلسه‌ی دوساعته روی کدی که داوطلب تا حالا ندیده است، با این تفاوت که این بار انتظار داریم از یک دستیار هوش مصنوعی کمک بگیرد. مهم نیست کار را تمام می‌کند یا نه. مهم این است که در کدی ناآشنا چطور با دستیار کار می‌کند و آیا می‌فهمد دستیار کجا اشتباه می‌کند.

چرا این کد

MiA یک میان‌افزار کوچک اینترنت اشیا است. عامل‌ها (agent) داده‌ی دستگاه‌ها را روی MQTT می‌فرستند و کاربر از طریق HTTP آن را می‌خواند و دستگاه را کنترل می‌کند. کل پروژه حدود ۱۹۰۰ خط پایتون در ۷۱ فایل است و شش تست دارد که همه پاس می‌شوند.

کد آن‌قدر کوچک است که در یک ساعت بشود خواندش، ولی روی چند قابلیت پایتون بنا شده که بیشتر ما درست یادمان نیست: یک metaclass که هر زیرکلاس `Thing` را با نام ثبت می‌کند؛ چند descriptor با اثر جانبی (مقداردهی به `Setting` یک پیام MQTT می‌فرستد و مقداردهی به `State` در Mongo می‌نویسد)؛ کنترلرهای singleton که وابستگی‌هایشان در `main.py` به‌صورت class attribute تنظیم می‌شود؛ و بارگذاری نوع دستگاه‌ها در لحظه‌ی نیاز با `importlib` بر اساس نام ماژول. دستیار هوش مصنوعی همه‌ی این‌ها را روان توضیح می‌دهد و در جزئیات اشتباه می‌کند. دقیقاً همین را می‌خواهیم.

پروژه یک محیط اجرای واقعی هم دارد: بروکر MQTT، پایگاه داده‌ی Mongo، یک عامل آزمایشی و یک API. یعنی داوطلب می‌تواند حرف دستیار را عملاً امتحان کند، نه این‌که فقط کد را بخواند. این‌که امتحان می‌کند یا نه، مهم‌ترین چیزی است که در این جلسه می‌بینیم.

شاخه‌ی `main` در کامیت `f0bd540` پنج باگ پنهان دارد که CI هیچ‌کدام را نمی‌گیرد. همه‌ی آن‌ها را پیش از نوشتن این جلسه با همان نسخه‌های قفل‌شده‌ی کتابخانه‌ها بازتولید کرده‌ایم؛ هیچ‌کدام ساختگی نیست.

باگ	کجا	اثر	تسک
تعداد آرگومان‌های callback در paho-mqtt 2.x	`I1820/mqtt/route.py`	نخ شبکه هنگام CONNACK می‌میرد و هیچ topicی subscribe نمی‌شود	۰
timestamp پیش‌فرض ثابت	`I1820/domain/log.py`	همه‌ی لاگ‌های عامل آزمایشی یک timestamp دارند: لحظه‌ی import	۰
فهرست فیلدها بین همه‌ی نوع‌ها مشترک است	`I1820/things/base.py`	هر نوع، فیلدهای همه‌ی نوع‌های بارشده را گزارش می‌کند	۲
`last_seen` به‌روز نمی‌شود	`I1820/discovery/discovery.py`	زمان عامل روی اولین ping می‌ماند	۳
خواندن settingی که هنوز مقدار ندارد	`I1820/things/fields/setting.py`	`KeyError` به HTTP 500 تبدیل می‌شود	۳

قالب جلسه

فردی، ۱۲۰ دقیقه چهار تسک زمان‌دار و یک تسک اضافه، بعد ۱۵ دقیقه جمع‌بندی.

هوش مصنوعیِ خودش هر ابزاری آزاد است؛ Claude Code، Cursor، Copilot یا حتی یک پنجره‌ی چت. یادداشت کنید کدام ابزار بود و آیا خودش می‌توانست دستور اجرا کند. با پنجره‌ی چت، داوطلب مجبور است خودش دست‌به‌کار شود؛ با ابزار agentic سؤال این می‌شود که آیا داوطلب می‌خواند ابزار چه کرده است.

با ناظر یک مصاحبه‌گر با برگه‌ی یادداشت و ضبط صفحه. اگر ناظر ندارید، متن گفت‌وگو با هوش مصنوعی و تاریخچه‌ی git بیشترِ چیزهای لازم را نگه می‌دارند.

کامیت بعد از هر تسک داوطلب انتهای هر تسک را تگ می‌کند (`task-0`، `task-1`، …). کار ناتمام اشکالی ندارد و خودش یک شاهد است.

تسک‌ها یکی‌یکی داده می‌شوند تسک‌های بعدی به باگ‌هایی اشاره می‌کنند که شاید در تسک‌های قبلی رو شده باشند. اگر زودتر بدهید، نتیجه خراب می‌شود.

محیط از قبل آماده است ایمیج‌ها pull شده و وابستگی‌ها نصب شده‌اند. زمانی که صرف راه‌اندازی می‌شود چیزی به ما نمی‌گوید.

آماده‌سازی

یک fork با نامی خنثی (مثلاً `mia-challenge`) بسازید. کل آماده‌سازی کمتر از یک ساعت وقت می‌گیرد.

- [ ] از کامیت `f0bd540` fork کنید و پوشه‌ی `.github/` را حذف کنید تا کسی به badge سبز CI اعتماد نکند. README را نگه دارید؛ پروتکل MQTT را توضیح می‌دهد و بخشی از آزمون است.
- [ ] باگ paho را درست **نکنید**. تسک ۰ به آن وابسته است.
- [ ] شمارنده‌ی عامل آزمایشی را افزایشی کنید تا «آخرین مقدار» معنی داشته باشد. فراخوانی `I1820Log(...)` را بدون timestamp نگه دارید؛ تله‌ی دوم همین است.
- [ ] ایمیج‌های `mongo:5.0` و `emqx/emqx:5.0.0` را pull کنید و یک محیط پایتون ۳.۱۲ با وابستگی‌های قفل‌شده آماده کنید.

- [ ] روی ماشین آماده‌سازی مطمئن شوید تله کار می‌کند: `main.py` و `dummy.py` را اجرا کنید. `GET /agents` باید `{}` برگرداند و traceback خطای `TypeError` مربوط به paho باید در ترمینال سرور دیده شود.
- [ ] این راهنما را چاپ کنید و صورت هر تسک را روی یک کارت جدا بنویسید.

تغییر `dummy.py`:

```
-    i = 2
+    i = 0
     while True:
-        i = (i**2) % 1024
+        i += 1
         ping()
```

متنی که به داوطلب می‌دهید

بدون تغییر بخوانید: شما به تیمی می‌پیوندید که MiA، یک میان‌افزار کوچک اینترنت اشیا، را نگه‌داری می‌کند. عامل‌ها داده‌ی دستگاه‌ها را روی MQTT می‌فرستند و کاربران از طریق HTTP آن‌ها را می‌خوانند و کنترل می‌کنند. شما این کد را تا حالا ندیده‌اید. دو ساعت وقت دارید و یک دستیار هوش مصنوعی به انتخاب خودتان.

۱. هر قدر که می‌خواهید از هوش مصنوعی استفاده کنید. برای ما مهم است چطور کار می‌کنید، نه این‌که از آن استفاده می‌کنید یا نه. ۲. بلند فکر کنید. بگویید از هوش مصنوعی چه می‌پرسید و چرا، و بعد از جوابش به چه نتیجه‌ای رسیده‌اید. ۳. بعد از هر تسک کامیت و تگ کنید (`git tag task-N`). ناتمام هم اشکالی ندارد. ۴. اگر چیزی در تسک مبهم بود، می‌توانید از ما بپرسید. شاید جواب بدهیم، شاید نه. ۵. متن گفت‌وگو با هوش مصنوعی را نگه دارید. آخر جلسه از شما می‌گیریم.

TASK 0 — اولین چراغ

۱۵ دقیقه

صورت مسئله برای داوطلب

سیستم را بالا بیاورید (`docker compose up -d`، بعد `main.py` و `dummy.py`). به ما نشان دهید که:

- `GET /agents` عاملی به نام `dummy` با یک دستگاه از نوع `dummy` را فهرست می‌کند.
- `POST /things` برای آن دستگاه مقدار `state` `chert` را برمی‌گرداند، و اگر چند ثانیه بعد دوباره صدا بزنید مقدار **بزرگ‌تری** برمی‌گرداند.

تله‌ی ۱ · سرور می‌گوید بالاست، ولی چیزی نمی‌شنود. handler برای امضای callback نسخه‌ی x.1 کتابخانه‌ی paho نوشته شده، ولی client با `CallbackAPIVersion.VERSION2` ساخته می‌شود. هنگام CONNACK، paho تابع `on_connect` را با پنج آرگومان صدا می‌زند، متد فقط چهار تا می‌پذیرد و نخ شبکه با این خطا می‌میرد:

```
TypeError: Handler.on_connect() takes 5 positional arguments but 6 were given
```

traceback لابه‌لای لاگ‌های rich چاپ می‌شود، بعد «Mia is up and running» می‌آید و سرور HTTP عادی کار می‌کند. `GET /agents` جواب `{}` می‌دهد. هیچ topic‌ی هرگز subscribe نمی‌شود.

تله‌ی ۲ · مقدار هیچ‌وقت جلو نمی‌رود. سازنده‌ی `I1820Log` آرگومان پیش‌فرض `timestamp=datetime.utcnow()` دارد که فقط یک بار، در لحظه‌ی import، محاسبه می‌شود. عامل آزمایشی هیچ‌وقت timestamp نمی‌فرستد، پس همه‌ی لاگ‌هایش یک زمان دارند. `find_one(sort=time desc)` در Mongo بین مقادیر مساوی یکی را برمی‌گرداند و «آخرین» دیگر معنایی ندارد. تا داوطلب این را نفهمد، معیار دوم برآورده نمی‌شود.

به چه چیزی دقت کنید:

- ترمینال سرور را می‌خواند، یا فقط می‌نویسد «agents خالی است» و traceback را به هوش مصنوعی نمی‌دهد؟
- وقتی هوش مصنوعی شبکه‌ی tenant، Docker یا بروکر را مقصر می‌داند، اول فرضیه را امتحان می‌کند یا تغییر پیشنهادی را همان‌طور اعمال می‌کند؟
- برای تله‌ی ۲، هوش مصنوعی معمولاً پیشنهاد می‌دهد به‌جای `time` روی `_id` مرتب کنید. این فقط نشانه را می‌پوشاند، ولی کار می‌کند. داوطلب می‌پرسد **چرا** timestampها مساوی‌اند، یا همان وصله را قبول می‌کند؟
- بعد از هر اصلاح، کل مسیر را دوباره اجرا می‌کند، یا با اولین جواب غیرخالی می‌گوید تمام شد؟

TASK 1 — مسیر یک لاگ

۱۵ دقیقه

صورت مسئله برای داوطلب

در حداکثر ده خط بنویسید از لحظه‌ای که یک عامل روی `I1820/main/log/send` پیام می‌فرستد تا لحظه‌ای که کاربر آن مقدار را با `POST /things` می‌خواند، چه اتفاقی می‌افتد. در هر مرحله نام فایل و تابع را بنویسید. بعد به این دو سؤال جواب دهید: مقدار بین این دو لحظه کجا نگه داشته می‌شود؟ و اگر عامل هیچ‌وقت ping نفرستاده بود چه می‌شد؟

این تسک تله ندارد. هدفش این است که ببینیم داوطلب برای فهمیدن کد چطور از هوش مصنوعی استفاده می‌کند، و ضمناً او را برای تسک ۲ که به فهم metaclass نیاز دارد آماده می‌کند. مسیر درست:

۱. `Handler.on_log` در `mqtt/route.py` ورودی را با `I1820Log.from_json` و بر اساس `log.json` اعتبارسنجی می‌کند.
۲. `Things.get(type)` در `things/base.py` ماژول `I1820.things.models.<type>` را همان لحظه import می‌کند و metaclass همان‌جا کلاس را ثبت می‌کند.
۳. `get_thing(agent, device)` نمونه را در `registered_things` پیدا می‌کند؛ این dict را `DiscoveryService.ping` پر کرده است. بدون `ThingNotFoundException`، ping رخ می‌دهد و لاگ با یک warning دور ریخته می‌شود.
۴. `setattr(thing, name, {...})` به `State.__set__` می‌رسد. آن هم singleton `LogController` را صدا می‌زند که `log_service`اش در `main.py` به‌صورت class attribute تنظیم شده و در نهایت به `MongodbLogAppender.create` می‌رسد.
۵. در خواندن، `getattr` به `State.__get__` می‌رسد که با `retrieve_last` از Mongo می‌خواند. مقدار فقط در Mongo است؛ نمونه‌ی thing چیزی نگه نمی‌دارد.

به چه چیزی دقت کنید:

- توضیح هوش مصنوعی را همان‌طور قبول می‌کند، یا از آن شماره‌ی خط می‌خواهد و فایل‌ها را خودش باز می‌کند؟
- اشتباه‌های رایج هوش مصنوعی در این تسک: می‌گوید state روی نمونه cache می‌شود؛ `Things.get` را خواندن یک فایل registry توصیف می‌کند؛ نمی‌بیند که `LogController` یک singleton است که در `main.py` تنظیم می‌شود. بشمارید داوطلب چند تا را می‌گیرد.
- برای مطمئن‌شدن، از محیط اجرا استفاده می‌کند؟ مثلاً `dummy.py` را متوقف کند و سرور را دوباره بالا بیاورد تا warning دورریختن لاگ را ببیند.

TASK 2 لامپ قابل تنظیم

۳۰ دقیقه

صورت مسئله برای داوطلب

نوع دستگاه `dimmable_lamp` را اضافه کنید. هر چه `lamp` دارد، این هم دارد؛ به‌علاوه‌ی setting `brightness` (عدد صحیح، ۰ تا ۱۰۰) و statistic `power`. معیار پذیرش:

- `GET /models/dimmable_lamp` دقیقاً فیلدهای خودش را برمی‌گرداند: settingهای `on` و `brightness`، statistic `power`، و نه چیز دیگری.
- `PUT /things` با `brightness: 60` یک notification روی `I1820/main/configuration/request` می‌فرستد. با هر subscriber MQTT نشانش دهید.
- برای هر دو تست بنویسید.

تله · زیرکلاس درست است، ولی endpoint هنوز غلط جواب می‌دهد. `class DimmableLamp(Lamp)` با دو فیلد جدید همان چیزی است که باید باشد. ولی `AbstractThing.__new__` فیلدها را با `getattr(instance, field_name).append(value)` جمع می‌کند و فهرست‌هایی که به آن‌ها اضافه می‌کند همان‌هایی‌اند که روی خودِ Thing تعریف شده‌اند. یعنی همه‌ی نوع‌ها در یک فهرست شریک‌اند. وقتی `curtain`، `lamp` و `temperature` import شده باشند، endpoint این را برمی‌گرداند:

```
{"type": "dimmable_lamp", "settings": ["on", "height", "brightness"],
 "states": ["temperature"], "statistics": ["power"], "events": []}
```

همین باگ `POST /things` با فهرست `states` خالی را روی هر actuatorی خراب می‌کند، چون handler بعدش `getattr(lamp, "temperature")` را صدا می‌زند و ۵۰۰ می‌دهد. اصلاح درست در metaclass است: هر کلاس فهرست‌های خودش را داشته باشد که با فیلدهای کلاس‌های پایه شروع می‌شوند. فیلترکردن در `to_dict` فقط نشانه را برای یک endpoint می‌پوشاند.

تله‌ی کوچک‌تر: فایل باید `models/dimmable_lamp.py` باشد، چون `Things.get` با نام ماژول import می‌کند و بعد نام کلاس را به‌شکل snake_case جست‌وجو می‌کند. `dimmablelamp.py` بدون خطا import می‌شود و بعد یک `ThingTypeNotImplementedException` گمراه‌کننده می‌دهد.

به چه چیزی دقت کنید:

- وقتی هوش مصنوعی گفت تمام شد، واقعاً `GET /models/dimmable_lamp` را صدا می‌زند؟ در بیشتر موارد هوش مصنوعی کلاس را می‌نویسد، شاید تستی برای `issubclass` هم بنویسد، و همان‌جا متوقف می‌شود.
- وقتی فیلدهای اضافه را دید، نوع‌های قبلی را هم بررسی می‌کند یا فقط نوع جدید را درست می‌کند؟
- ریشه یا نشانه؟ اصلاح metaclass، فیلتر در `to_dict`، یا یک فهرست hardcode؟
- اعتبارسنجی بازه‌ی brightness کجا می‌رود؟ هوش مصنوعی معمولاً آن را در handler HTTP می‌گذارد. داوطلب می‌پرسد descriptor جای بهتری نیست، وقتی `Setting.__set__` پیش از هر اعتبارسنجی پیام را می‌فرستد؟
- برای تست notification، فایل `test_thing.py` الگوی `DummyMQTTService` را نشان می‌دهد. هوش مصنوعی آن را پیدا می‌کند یا یک mock تازه می‌سازد؟ داوطلب متوجه تکرار می‌شود؟

TASK 3 دو گزارش باگ

۲۵ دقیقه

صورت مسئله برای داوطلب

دو تیکت از تیم عملیات. هر دو را درست کنید و برای هر کدام تست رگرسیونی بنویسید که پیش از اصلاح شکست بخورد و بعد از آن پاس شود.

- **OPS-118**: «فیلد `time` در `GET /agents` برای عاملی که ساعت‌هاست ping می‌فرستد هیچ‌وقت عوض نمی‌شود. برای همین alert عامل‌های stale ما هیچ‌وقت فعال نمی‌شود.»
- **OPS-121**: «خواندن یک لامپ با `POST /things` و `settings: ["on"]` اگر هنوز کسی آن را روشن یا خاموش نکرده باشد HTTP 500 می‌دهد. جواب تمیزی می‌خواهیم.»

تله · دو خط غلط، کنار خط درست. در `DiscoveryService.ping`، در شاخه‌ی pingهای بعدی، `agent.last_seen = now` روی شیء ورودی انجام می‌شود که بلافاصله دور ریخته می‌شود. عامل ذخیره‌شده همان timestamp اول را نگه می‌دارد. دو خط بالاتر هم `if self._agents[agent.ident] == agent.things` هست که یک `Agent` را با یک `set` مقایسه می‌کند و همیشه نادرست است. هوش مصنوعی اغلب همین مقایسه را «درست» می‌کند و می‌گوید تمام شد، در حالی که `last_seen` هنوز کهنه است. تست موجود `test_discovery.py` در تمام این مدت پاس می‌شود.

در `Setting.__get__`، OPS-121 بدون هیچ محافظی از dict می‌خواند. README می‌گوید settingها باید پیش از خواندن مقدار گرفته باشند، پس این‌جا یک تصمیم واقعی درباره‌ی قرارداد API وجود دارد: `null` برگردانیم مثل `Statistic`، یا ۴۰۴، یا ۴۰۰. داوطلب باید بفهمد این تصمیم فقط مال او نیست. اگر `None` برگرداند، نوع بازگشتی هم باید `T | None` شود، وگرنه mypy در CI شکست می‌خورد.

به چه چیزی دقت کنید:

- تست جدید را **پیش از** اصلاح اجرا می‌کند تا شکستش را ببیند؟ تستی که هوش مصنوعی می‌نویسد و دو ping پشت هم بدون جلو بردن زمان می‌زند، قبل و بعد از اصلاح پاس می‌شود.
- متوجه می‌شود که هوش مصنوعی خط اشتباهی را عوض کرده؟ چطور: با خواندن diff، با اجرای سرور، یا اصلاً متوجه نمی‌شود؟
- سؤال قرارداد API را با مصاحبه‌گر مطرح می‌کند، یا می‌گذارد هوش مصنوعی بی‌سروصدا تصمیم بگیرد؟
- اصلاح OPS-121 در descriptor انجام می‌شود، یا هوش مصنوعی handler را در try/except می‌پیچد و `KeyError` بعدی را هم پنهان می‌کند؟

TASK 4 چند دستگاه (اضافه)

اگر وقت ماند

صورت مسئله برای داوطلب

`POST /things` فهرستی از `device_id` می‌پذیرد، ولی یک شیء تخت برمی‌گرداند و فقط مقادیر آخرین دستگاه در آن می‌ماند. کاری کنید نتیجه به تفکیک دستگاه برگردد، بدون این‌که فراخوانی تک‌دستگاهی خراب شود.

این تسک بیشتر طراحی است تا باگ‌یابی، و تله‌ای ندارد. می‌خواهیم ببینیم داوطلب سازگاری با نسخه‌ی قبلی را در نظر می‌گیرد و درباره‌ی شکل پاسخ سؤال می‌کند، یا می‌گذارد هوش مصنوعی یکی را انتخاب کند. برای پرکردن وقت باقی‌مانده و دیدن رفتار او وقتی جواب «درستی» وجود ندارد که بشود با آن سنجید.

سنجش: با همان چهار شایستگی

هر تسک با **همان چهار شایستگی** و **همان مقیاس** سند ارزیابی سنجیده می‌شود: **ضعیف · مرزی · قوی · سنجیده نشد**. هوش مصنوعی شایستگی‌ها را عوض نمی‌کند؛ فقط شاهدهایی که برای هر شایستگی می‌شماریم عوض می‌شوند.

شایستگی	ضعیف	قوی
داده‌ساختار، الگوریتم و حل مسئله	همان‌جایی که نشانه دیده می‌شود وصله می‌زند (_id، فیلتر در to_dict)؛ در جمع‌بندی نمی‌تواند بگوید state کجا نگه داشته می‌شود یا metaclass چه می‌کند	ریشه‌ی هر باگ را پیدا می‌کند و اثرش را روی جاهای دیگر هم بررسی می‌کند؛ مسیر داده را بدون کمک می‌کشد
کدنویسی	می‌گذارد هوش مصنوعی metaclass را بازنویسی کند، pydantic اضافه کند یا به FastAPI مهاجرت کند؛ تستش پیش از اصلاح هم پاس می‌شود	تغییرات را محدود به تسک نگه می‌دارد؛ اول تستی می‌نویسد که شکست بخورد؛ الگوهای موجود در مخزن را دنبال می‌کند
ارتباط مؤثر	پرامپت‌هایی مثل «agents خالیه، چرا»؛ توضیح هوش مصنوعی را همان‌طور قبول می‌کند؛ قرارداد API را هوش مصنوعی بی‌سروصدا تعیین می‌کند	پرامپتش traceback، نام فایل و انتظارش را دارد؛ توضیح غلط را با شاهد رد می‌کند؛ تصمیم طراحی را صریح مطرح می‌کند و می‌پرسد یا فرضش را می‌گوید
سرعت و به‌نتیجه‌رسانی	پیشنهادهای هوش مصنوعی را یکی‌یکی امتحان می‌کند تا یکی کار کند؛ هیچ‌وقت endpoint را صدا نمی‌زند یا تست را اجرا نمی‌کند	پیش از اصلاح مشکل را بازتولید و بعد از آن نتیجه را بررسی می‌کند، diff را می‌خواند، و باز هم در زمان تعیین‌شده به کد کارکننده می‌رسد

برای هر تسک سه شمارش ساده هم ثبت کنید: **فایل‌هایی که داوطلب باز کرد**، **فایل‌هایی که هوش مصنوعی باز کرد**، و **ادعاهای هوش مصنوعی که بدون بررسی پذیرفته شد**. این سه عدد، «کد می‌خواند» را از «درباره‌ی کد می‌خواند» بهتر از هر نمره‌ای جدا می‌کنند.

جمع‌آوری داده و جمع‌بندی

چه چیزهایی را جمع کنید تاریخچه‌ی git با تگ‌های task-N و working tree پایانی؛ متن گفت‌وگو با هوش مصنوعی (خروجی جلسه‌ی Claude Code، چت Cursor، یا متن پنجره‌ی چت)؛ ضبط صفحه همراه با صدای داوطلب؛ برگه‌ی مصاحبه‌گر با یادداشت‌های زمان‌دار.

جمع‌بندی، ۱۵ دقیقه

۱. مسیر یک پیام لاگ را از عامل تا Mongo بکش. کدام فایل نوع دستگاه را ثبت می‌کند، و کِی؟
۲. امروز کدام حرف‌های هوش مصنوعی غلط از آب درآمد؟ چطور فهمیدی؟
۳. جایی بود که اعتمادت به هوش مصنوعی را از دست بدهی؟ چه چیزی باعثش شد؟
۴. کدام تغییر را با کمترین اطمینان merge می‌کنی، و چرا؟
۵. دو سرور MiA با tenant متفاوت روی یک بروکر مشترک‌اند. چه چیزی هنوز بینشان مشترک می‌ماند؟ (پاسخ: روی MQTT هیچ؛ registry نوع‌ها و singletonهای کنترلر هم per-process هستند. داوطلبی که فکر می‌کند و همین را می‌گوید، مدل درستی از سیستم دارد.)
۶. دوست داشتی هوش مصنوعی چه چیزی را می‌دید یا می‌توانست انجام بدهد که نمی‌توانست؟

راه‌حل‌های مرجع

نسخه‌ی خلاصه برای مصاحبه‌گر. هر پنج اصلاح را با هم روی یک کپی از مخزن اعمال کرده‌ایم: شش تست موجود و mypy پاس می‌شوند و هر یک از باگ‌های جدول به رفتار درست برمی‌گردد.

تسک ۰ · callback در paho

```
# I1820/mqtt/route.py
def on_connect(self, client, userdata, flags, reason_code, properties):
    ...
```

تسک ۰ · timestamp ثابت

```
# I1820/domain/log.py
def __init__(self, type, device, states, agent, timestamp=None):
    ...
    self.timestamp = timestamp or datetime.datetime.now(datetime.UTC)
```

تسک ۲ · فهرست فیلد جدا برای هر کلاس

```
# I1820/things/base.py, inside AbstractThing.__new__ after the abstract check
for attr in ("statistics", "settings", "events", "states"):
    inherited = [f for base in bases for f in getattr(base, attr, [])]
    setattr(instance, attr, list(inherited))
for _, value in namespace.items():
    if isinstance(value, Field):
        getattr(instance, value.field_name).append(value)
```

تسک ۳ · last_seen

```
# I1820/discovery/discovery.py, in the else branch of ping()
stored = self._agents[agent.ident]
stored.last_seen = datetime.datetime.now(datetime.UTC)
if stored.things == agent.things:
    return
```

تسک ۳ · setting بدون مقدار

```
# I1820/things/fields/setting.py
def __get__(self, obj: Thing, _) -> T | None:
    return self.storage.get((obj.agent_id, obj.device_id))
```

برگرداندن None با رفتار Statistic و شکل پاسخ handler خواندن هم‌خوان است. ۴۰۴ هم اگر تیم ترجیح بدهد قابل دفاع است. مهم این است که این رفتار انتخاب شده باشد، نه این‌که از یک KeyError به ارث رسیده باشد.

ملاحظات

- پنج باگ در محیط پایتون ۳.۱۲ با نسخه‌های قفل‌شده بازتولید شده‌اند (pymongo 4.10.1، sanic 24.6.0، paho-mqtt 2.1.0). خرابی MQTT روی یک بروکر درون‌فرایندی بازتولید شد، نه روی EMQX.
- این‌که Mongo بین timestampهای مساوی دقیقاً کدام را برمی‌گرداند بررسی نشده است. تله همچنان برقرار است، چون معیار «مقدار بزرگ‌تر در فراخوانی دوم» به‌شکل قابل اعتماد برآورده نمی‌شود. با این حال رفتار واقعی را روی ماشین آماده‌سازی ببینید و اگر لازم شد صورت مسئله را تغییر دهید.
- زمان‌بندی با فرض محیط آماده است. اگر داوطلب مجبور شود ایمیج یا وابستگی نصب کند، ۲۰ دقیقه به تسک ۰ اضافه کنید و آن زمان را در ارزیابی حساب نکنید.